Title: PricePilot AI: Dynamic Pricing Optimization & Revenue Intelligence System

Install & Import Libraries

```
# Install Libraries
!pip -q install xgboost

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from xgboost import XGBRegressor

import warnings
warnings.filterwarnings("ignore")
```

```
# Load Datasets
cp = pd.read_csv("competitor_prices.csv")
ca = pd.read_csv("customer_activity.csv")
do = pd.read_csv("daily_operations.csv")
pc = pd.read_csv("product_catalog.csv")

print("Competitor:", cp.shape)
print("Customer:", ca.shape)
print("Operations:", do.shape)
print("Products:", pc.shape)
```

```
Competitor: (18848, 5)
Customer: (92668, 15)
Operations: (43245, 26)
Products: (69, 13)
```

Data Cleaning

```
# Convert dates
do["date"] = pd.to_datetime(do["date"])
cp["date"] = pd.to_datetime(cp["date"])
ca["timestamp"] = pd.to_datetime(ca["timestamp"])
pc["launch_date"] = pd.to_datetime(pc["launch_date"])

# Remove duplicate rows
do = do.drop_duplicates()
cp = cp.drop_duplicates()
ca = ca.drop_duplicates()
pc = pc.drop_duplicates()

# Fill customer activity missing values
ca["quantity"] = ca["quantity"].fillna(0)
ca["total_amount"] = ca["total_amount"].fillna(0)

# Fill competitor price missing values
cp["competitor_price"] = cp["competitor_price"].fillna(
    cp.groupby("product_id")["competitor_price"].transform("median")
)

# Fill operational competitor prices
do["competitor_avg_price"] = do["competitor_avg_price"].fillna(
    do["competitor_avg_price"].median()
)

do["competitor_min_price"] = do["competitor_min_price"].fillna(
    do["competitor_min_price"].median()
)
```

```
print("Cleaning completed")
```

```
Cleaning completed
```

```
# Checking Missing Values
print("Daily Operations Missing Values")
display(do.isnull().sum())

print("\nCustomer Activity Missing Values")
display(ca.isnull().sum())

print("\nCompetitor Prices Missing Values")
display(cp.isnull().sum())

print("\nProduct Catalog Missing Values")
display(pc.isnull().sum())
```

```
Daily Operations Missing Values
```

	0
date	0
product_id	0
category	0
list_price	0
discount_pct	0
final_price	0
units_sold	0
revenue	0
channel	0
stock_available	0
restocked	0
stockout_flag	0
warehouse	0
competitor_avg_price	0
competitor_min_price	0
day_of_week	0
is_weekend	0
is_holiday	0
holiday_name	42684
is_festival_season	0
festival_name	38484
is_promotional_event	0
promotional_event_name	39905
market_demand_index	0
inflation_rate_pct	0

EDA

```
# Sales distribution
plt.figure(figsize=(7,4))
sns.histplot(do["units_sold"], kde=True)
plt.title("Units Sold Distribution")
plt.show()

# Price vs demand
plt.figure(figsize=(7,4))
sns.scatterplot(
    data=do,
    x="final_price",
    y="units_sold"
)
plt.title("Price vs Demand")
plt.show()

# Sales trend
x = do.groupby("date")["units_sold"].sum()

plt.figure(figsize=(10,4))
plt.plot(x)
plt.title("Sales Trend")
plt.xlabel("Date")
plt.ylabel("Units Sold")
plt.show()

# Correlation
plt.figure(figsize=(10,6))
sns.heatmap(
    do.select_dtypes("number").corr(),
    annot=True,
    cmap="coolwarm"
```

```
)
plt.title("Correlation Matrix")
plt.show()
```

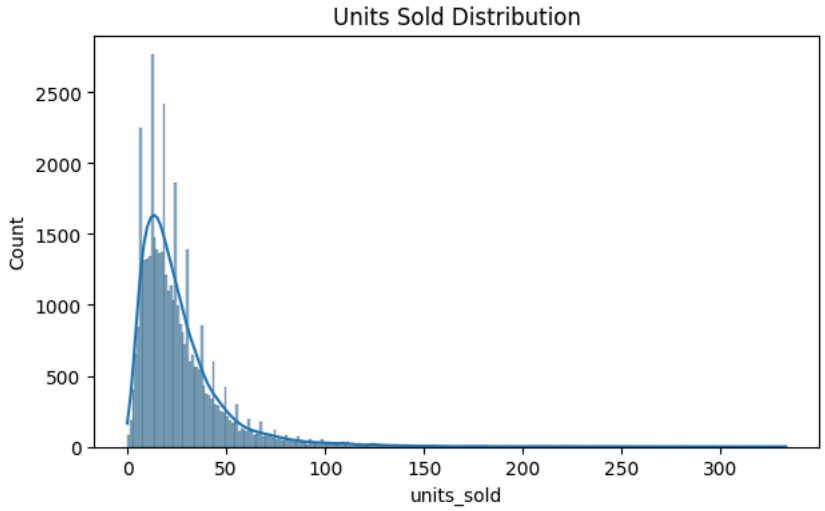
date	0
product_id	0
competitor_name	0
competitor_price	0
in_stock	0

dtype: int64

```
Product Catalog Missing Values
```

	0
product_id	0
product_name	0
category	0
subcategory	0
brand	0
cost_price	0
base_price	0
currency	0
launch_date	0
weight_kg	0
rating	0
num_reviews	0
is_active	0

dtype: int64



Price vs Demand

```
# Customer + Competitior features
ca["date"] = ca["timestamp"].dt.date
ca["date"] = pd.to_datetime(ca["date"])

cu = ca.pivot_table(
    index=["date", "product_id"],
    columns="event_type",
    values="event_id",
    aggfunc="count",
    fill_value=0
).reset_index()

cu = cu.rename(columns={
    "view": "views",
    "add_to_cart": "carts",
    "purchase": "purchases"
})

for c in ["views", "carts", "purchases"]:
    if c not in cu.columns:
        cu[c] = 0

co = cp.groupby(
    ["date", "product_id"]
).agg(
    ca=("competitor_price", "mean"),
    cm=("competitor_price", "min"),
    ci=("in_stock", "mean")
).reset_index()
```

```
# Combine all Datasets
df = do.merge(
    pc,
    on="product_id",
    how="left",
    suffixes=("", "_p")
)

df = df.merge(
    cu,
    on=["date", "product_id"],
    how="left"
)

df = df.merge(
    co,
    on=["date", "product_id"],
    how="left"
)

for c in ["views", "carts", "purchases", "ca", "cm", "ci"]:
    df[c] = df[c].fillna(0)
```

```
# Replace 0s in 'ca' with 'competitor_avg_price' where 'ca' is 0
df.loc[df["ca"] == 0, "ca"] = df["competitor_avg_price"]
# Replace 0s in 'cm' with 'competitor_min_price' where 'cm' is 0
df.loc[df["cm"] == 0, "cm"] = df["competitor_min_price"]

print(df.shape)
```

```
(43245, 44)
```

Feature Engineering

```
df["mo"] = df["date"].dt.month
df["dy"] = df["date"].dt.day
df["wk"] = df["date"].dt.isocalendar().week.astype(int)

df["pg"] = df["final_price"] - df["ca"]
df["pr"] = df["final_price"] / (df["ca"] + 1)

df["cr"] = df["carts"] / (df["views"] + 1)
df["br"] = df["purchases"] / (df["views"] + 1)

df["profit"] = (
    df["final_price"] - df["cost_price"]
) * df["units_sold"]

cat = [
    "category",
    "subcategory",
    "brand",
    "channel",
    "warehouse",
    "day_of_week"
]

for c in cat:
    le = LabelEncoder()
    df[c] = le.fit_transform(
        df[c].astype(str)
    )

fs = [
    "final_price",
    "list_price",
    "discount_pct",
    "stock_available",
    "competitor_avg_price",
    "competitor_min_price",
    "market_demand_index",
    "inflation_rate_pct",
    "consumer_confidence_index",
    "rating",
    "num_reviews",
    "weight_kg",
    "views",
    "carts",
    "purchases",
    "ca",
    "cm",
    "ci",
    "pg",
    "pr",
    "cr",
    "br",
    "is_weekend",
    "is_holiday",
    "is_festival_season",
    "is_promotional_event",
    "restocked",
    "stockout_flag",
    "mo",
    "dy",
    "wk"
] + cat

X = df[fs].copy()
```

```
y = df["units_sold"]

X = X.replace(
    [np.inf, -np.inf],
    np.nan
)

X = X.fillna(
    X.median(numeric_only=True)
)
```

Train/Test Split

```
df = df.sort_values("date").reset_index(drop=True)

X = df[fs].copy()
y = df["units_sold"]

X = X.replace(
    [np.inf, -np.inf],
    np.nan
)

X = X.fillna(
    X.median(numeric_only=True)
)

n = int(len(df) * 0.8)

x1 = X.iloc[:n]
x2 = X.iloc[n:]

y1 = y.iloc[:n]
y2 = y.iloc[n:]

print("Train:", x1.shape)
print("Test :", x2.shape)
```

```
Train: (34596, 37)
Test : (8649, 37)
```

Train Random Forest

```
rf = RandomForestRegressor(
    n_estimators=300,
    max_depth=20,
    min_samples_split=5,
    random_state=42,
    n_jobs=-1
)

rf.fit(x1, y1)

p1 = rf.predict(x2)

m1 = mean_absolute_error(y2, p1)
r1 = np.sqrt(mean_squared_error(y2, p1))
s1 = r2_score(y2, p1)

print("Random Forest")
print("MAE :", m1)
print("RMSE:", r1)
print("R2  :", s1)
```

```
Random Forest
MAE : 6.479054243083431
RMSE: 10.035266642477405
R2  : 0.8816069986609657
```

Train XGBoost

```
xg = XGBRegressor(
    n_estimators=500,
    max_depth=8,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1
)

xg.fit(x1, y1)

p2 = xg.predict(x2)

m2 = mean_absolute_error(y2, p2)
r2 = np.sqrt(mean_squared_error(y2, p2))
s2 = r2_score(y2, p2)

print("XGBoost")
print("MAE :", m2)
print("RMSE:", r2)
print("R2  :", s2)
```

```
XGBoost
MAE : 5.816123008728027
RMSE: 8.762190256344523
R2  : 0.9097403883934021
```

```
# Select Best Model
res = pd.DataFrame({
    "Model": ["Random Forest", "XGBoost"],
    "MAE": [m1, m2],
    "RMSE": [r1, r2],
    "R2": [s1, s2]
})

display(res)

if r2 < r1:
    bm = xg
    bn = "XGBoost"
    bp = p2
else:
    bm = rf
    bn = "Random Forest"
    bp = p1

print("Best Model:", bn)
```

	Model	MAE	RMSE	R2
0	Random Forest	6.479054	10.035267	0.881607
1	XGBoost	5.816123	8.762190	0.909740

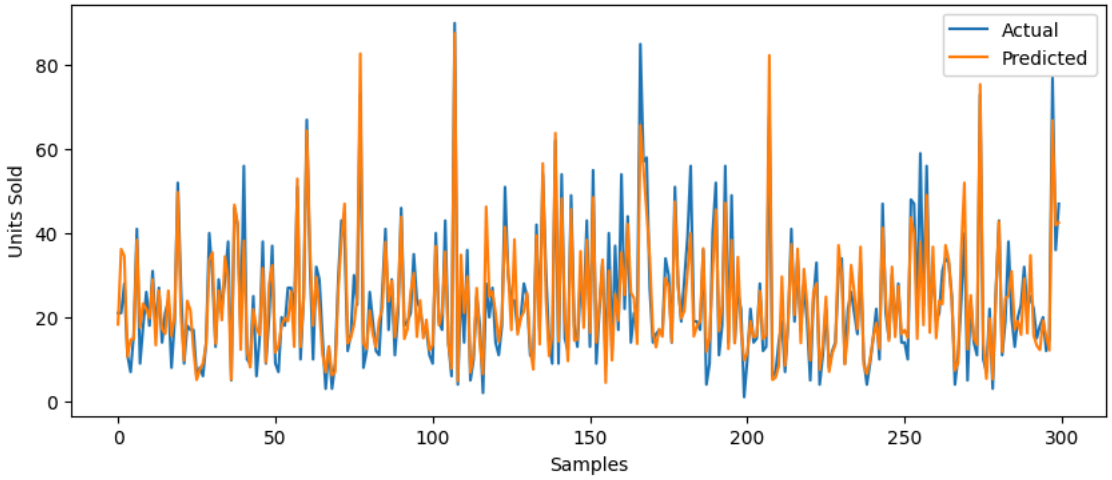
```
Best Model: XGBoost
```

```
# Actual vs Predicted
plt.figure(figsize=(10,4))

plt.plot(
    y2.values[:300],
    label="Actual"
)

plt.plot(
    bp[:300],
    label="Predicted"
)

plt.title("Actual vs Predicted Demand")
plt.xlabel("Samples")
plt.ylabel("Units Sold")
plt.legend()
plt.show()
```

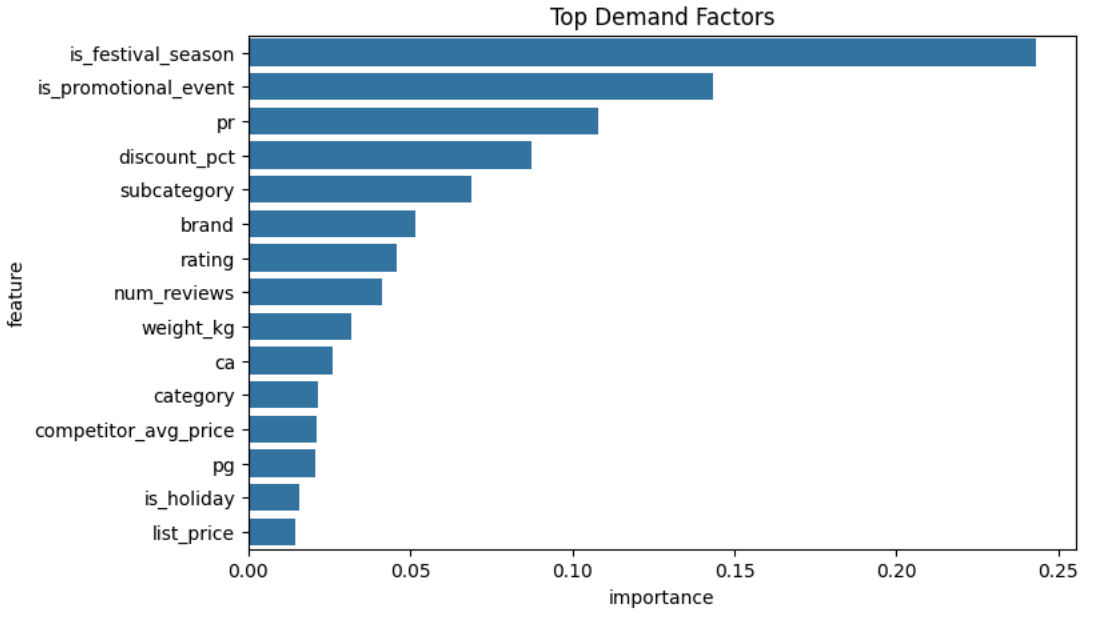



```
# Feature Importance
fi = pd.DataFrame({
    "feature": fs,
    "importance": bm.feature_importances_
})

fi = fi.sort_values(
    "importance",
    ascending=False
).head(15)

plt.figure(figsize=(8,5))
sns.barplot(
    data=fi,
    x="importance",
    y="feature"
)

plt.title("Top Demand Factors")
plt.show()
```




```
# Demand Prediction
pr = df.iloc[n:].copy()

pr["pred_demand"] = np.maximum(bp, 0)

pr["pred_revenue"] = (
    pr["pred_demand"] *
```

```
    pr["final_price"]
)

pr["pred_profit"] = (
    pr["pred_demand"] *
    (pr["final_price"] - pr["cost_price"])
)

display(
    pr[
        [
            "date",
            "product_id",
            "product_name",
            "final_price",
            "pred_demand",
            "pred_revenue",
            "pred_profit"
        ]
    ].head(20)
)
```

	date	product_id	product_name	final_price	pred_demand	pred_revenue	pred_profit
34596	2025-08-28	P1060	Orbis Hotel Room - Deluxe Max	227.68	18.316334	4170.262873	2114.071244
34597	2025-08-28	P1056	Solace Hotel Room - Standard Pro	37.36	36.216236	1353.038581	157.178465
34598	2025-08-28	P1033	Lumen Face Serum Lite	140.63	34.569321	4861.483567	308.358340
34599	2025-08-28	P1040	Lumen Moisturizer Pro	627.43	10.659793	6688.273859	3818.124621
34600	2025-08-28	P1009	Solace Laptop Sleeve Lite	411.39	14.672592	6036.157690	2122.977360
34601	2025-08-28	P1036	Vertex Perfume Pro	405.31	14.828543	6010.156646	1860.685539
34602	2025-08-28	P1047	Urbana Dumbbell Set Max	305.17	38.431435	11728.120906	568.785233
34603	2025-08-28	P1023	Crestline Air Fryer Lite	355.25	17.518400	6223.411668	574.603526
34604	2025-08-28	P1034	Vertex Hair Dryer Pro	274.91	23.237089	6388.108180	1571.524340
34605	2025-08-28	P1014	Vertex Jacket Pro	155.31	21.975414	3413.001591	755.514743
34606	2025-08-28	P1037	Aether Perfume Lite	143.44	19.726343	2829.546662	698.509811
34607	2025-08-28	P1016	Vertex Handbag Pro	314.99	28.990232	9131.633325	1849.286929
34608	2025-08-28	P1027	Urbana Vacuum Cleaner Pro	124.02	13.354606	1656.238196	435.493691
34609	2025-08-28	P1000	Nova Headphones Pro	250.75	26.348166	6606.802502	2577.114069
34610	2025-08-28	P1053	Pinnacle Backpack Pro	191.62	17.169441	3290.008327	1201.517497
34611	2025-08-28	P1018	Orbis Sunglasses Pro	602.33	16.040972	9661.958518	4761.281237
34612	2025-08-28	P1029	Aether Vacuum Cleaner Max	33.11	26.339645	872.105659	356.638799
34613	2025-08-28	P1059	Lumen Hotel Room - Deluxe Lite	34.01	15.560397	529.209107	212.710629
34614	2025-08-28	P1004	Pinnacle Bluetooth Speaker Pro	171.10	20.316370	3476.130909	165.578416
34615	2025-08-28	P1045	Pinnacle Dumbbell Set Pro	408.81	49.788422	20354.004647	969.380569

```
# Optimal Price Prediction
def opt(row):

    cur = row["final_price"]
    cost = row["cost_price"]

    ps = np.arange(
        max(cost * 1.05, cur * 0.8),
        cur * 1.2,
        1
    )

    z = pd.DataFrame([row] * len(ps))

    z["final_price"] = ps
    z["pg"] = z["final_price"] - z["ca"]
    z["pr"] = z["final_price"] / (z["ca"] + 1)

    z = z[fs]
```

```
    z = z.replace(
        [np.inf, -np.inf],
        np.nan
    ).fillna(0)

    d = np.maximum(
        bm.predict(z),
        0
    )

    pf = (ps - cost) * d

    k = np.argmax(pf)

    return ps[k], d[k], ps[k] * d[k], pf[k]
```

```
# Generate Final Pricing Recommendations
s = df.tail(500)

a = []

for _, row in s.iterrows():

    op, dm, rv, pf = opt(row)

    a.append([
        row["date"],
        row["product_id"],
        row["product_name"],
        row["category"],
        row["final_price"],
        row["cost_price"],
        row["ca"],
        op,
        dm,
        rv,
        pf
    ])

rec = pd.DataFrame(
    a,
    columns=[
        "date",
        "product_id",
        "product_name",
        "category",
        "current_price",
        "cost_price",
        "competitor_price",
        "optimal_price",
        "expected_demand",
        "expected_revenue",
        "expected_profit"
    ]
)

rec["change_pct"] = (
    (rec["optimal_price"] - rec["current_price"])
    / rec["current_price"]
) * 100

rec["recommendation"] = np.where(
    rec["change_pct"] > 3,
    "Increase Price",
    np.where(
        rec["change_pct"] < -3,
        "Decrease Price",
        "Keep Price"
    )
)

display(rec.head(20))
```